MINI PROJECT REPORT

THE LAST ESCAPE

A Pixel-Art-Inspired Mystery Survival Game

Submitted by	Parnika Lodha
UID	25BCD10078
Programme	BCA – Data Science
University	Chandigarh University
Section / Semester	25BCD-2A / Semester 3rd
Faculty Guide	Mr.Jitendra Kumar

Table of Contents

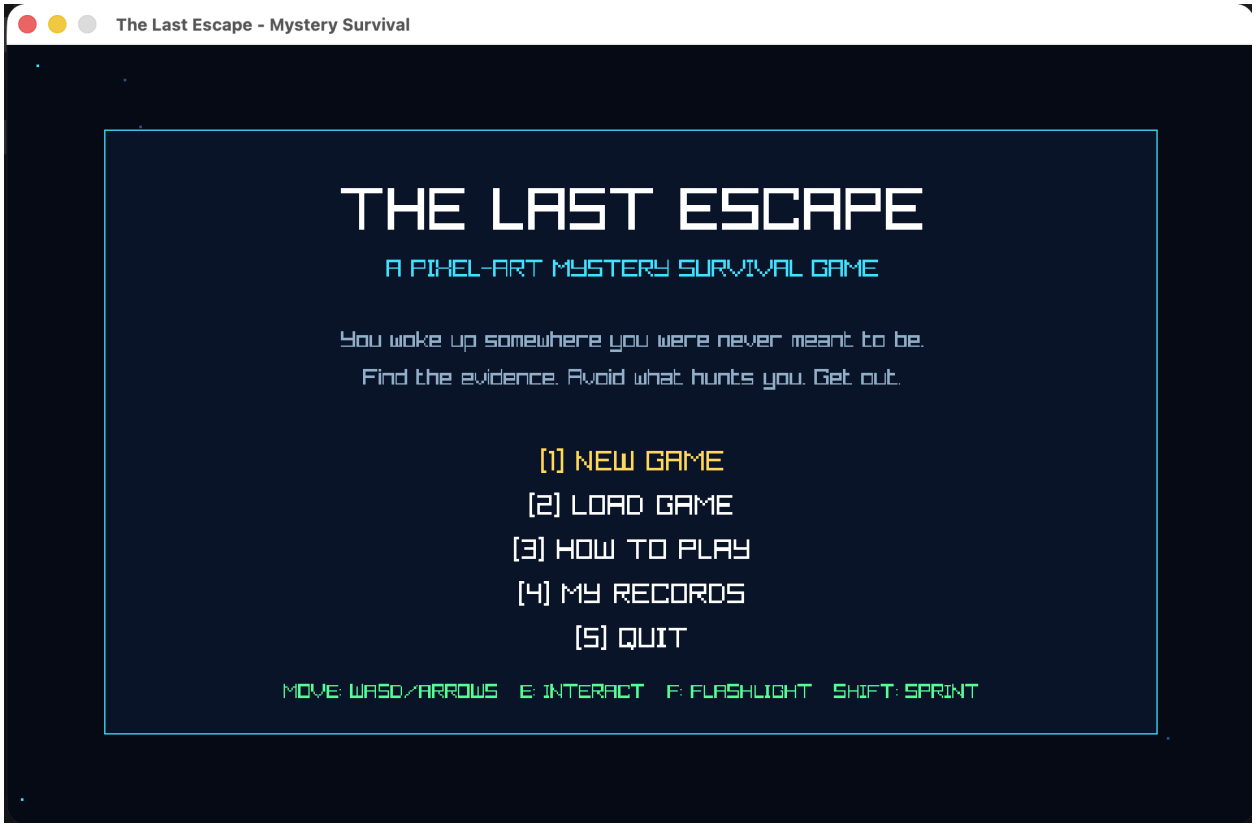
1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Tools and Technologies
7. System Requirements
8. Game Design and Gameplay
9. Controls
10. Object-Oriented Programming Concepts
11. System Workflow / Algorithm
12. Implementation Details
13. Build and Execution Procedure
14. Testing
15. Limitations
16. Future Enhancements
17. Conclusion
18. References

1. Abstract

This project, titled “The Last Escape,” is a 2D, top-down mystery-survival game developed in C++ using the Raylib graphics library. The player explores a dark facility made up of four connected rooms, collects clues and useful items, avoids a pursuing enemy, and attempts to reach the exit. The game includes a title menu, movement and interaction controls, an inventory screen, health and energy indicators, flashlight and sprint mechanics, save/load support, records, and multiple possible outcomes including a normal escape, being caught, and a secret ending.

The project demonstrates OOP-oriented programming through separate data structures for the player, enemies, items, and rooms, along with functions that manage world construction, movement, interaction, rendering, saving, and game-state changes. The project is an educational mini-project and can be extended with sprite animations, sound, more puzzles, and richer room designs.

2. Introduction



Games provide a practical way to apply programming concepts because they combine input handling, logic, data management, graphics, and state changes. “The Last Escape” uses a mystery-horror theme to make these concepts part of an interactive experience rather than a text-only program.

The game uses a top-down view and a pixel-art-inspired visual style. The player moves through connected rooms, searches for glowing objects, gathers clues, and avoids an enemy. The player must collect three clues and a security keycard before reaching the exit. Collecting hidden evidence can unlock a secret ending.

3. Problem Statement

Text-based programs can demonstrate basic logic, but they provide limited visual feedback and interaction. The aim of this project is to build a more engaging C++ application with a graphical window, keyboard controls, game objects, room navigation, player status, and persistent save data. The game also provides an example of how OOP concepts can be applied to a larger program.

4. Objectives

Develop a graphical game using C++ and Raylib.

Create a player that can move through multiple connected rooms.

Add items and clues that the player can collect through interaction.

Implement an enemy that follows the player and can cause damage.

Track player health, energy, inventory, and flashlight availability.

Provide a title menu, help screen, inventory screen, records screen, and ending screen.

Support saving and loading progress using local text files.

Demonstrate OOP-oriented program organization and provide a base for future improvements.

5. Scope of the Project

The current version focuses on a single-player, keyboard-controlled survival experience. It includes four rooms, room transitions, collectible items, clue collection, a security keycard, hidden evidence, an enemy, health and energy, sprinting, a flashlight toggle, saving/loading, records, and multiple outcomes. The visual assets are drawn with Raylib shapes and text, so no external sprite pack is required.

The current implementation is a student mini-project prototype rather than a commercial game. Detailed sprite animations, sound design, elaborate keypad puzzles, and advanced artificial intelligence are possible future extensions.

6. Tools and Technologies

Programming language: C++ (compiled with the C++17 standard).

Graphics library: Raylib 6.2, used for the window, drawing, keyboard input, timing, and graphics initialization.

Build tools: g++ and pkg-config.

Development environment: Visual Studio Code on macOS.

Development hardware: Apple Silicon MacBook Air. Raylib successfully initialized on the development machine during the initial run.

Persistence: local text files named last_escape_save.txt and last_escape_records.txt.

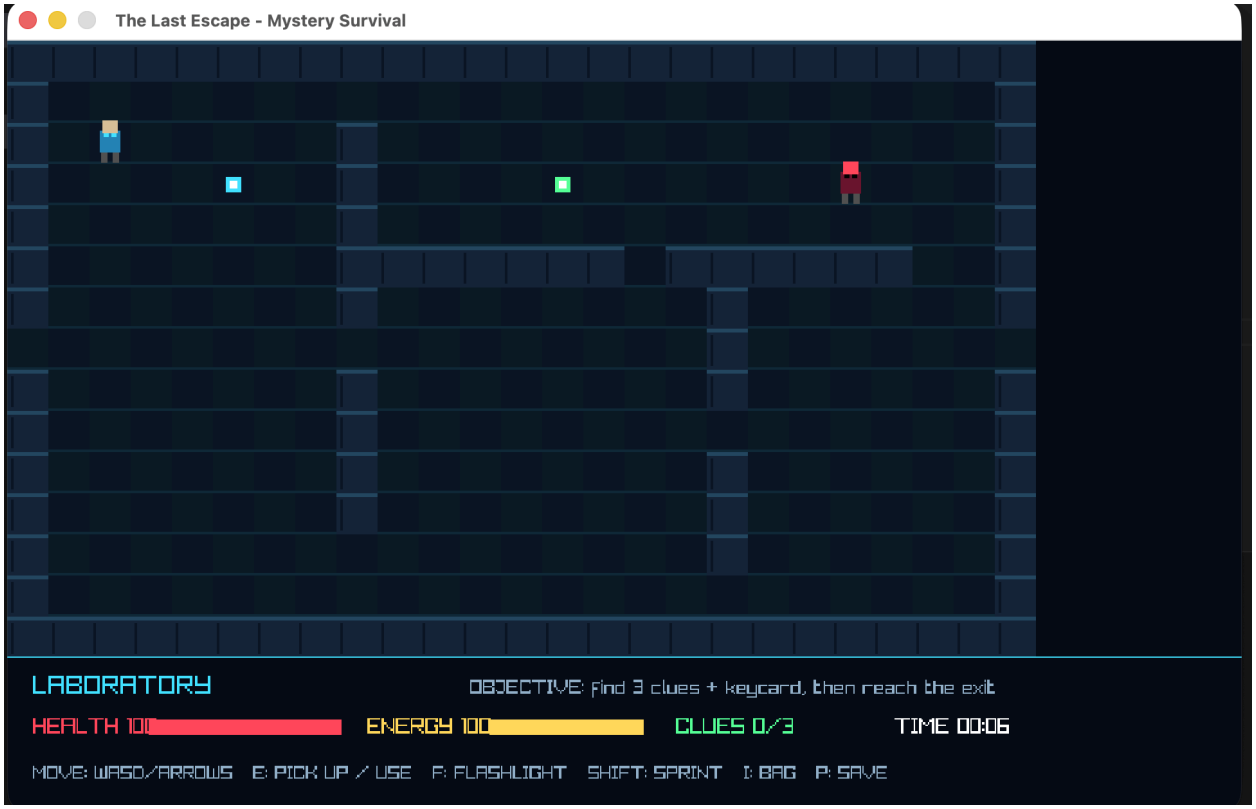
7. System Requirements

Hardware: a Mac or compatible desktop/laptop capable of running a graphical application, a keyboard, and a display.

Software: macOS, a C++ compiler such as g++, Homebrew, Raylib, pkg-config, and Visual Studio Code or another code editor.

Installation on macOS: run `brew install raylib pkg-config`, open the project folder in VS Code, and compile from the project directory using the command shown in the implementation section.

8. Game Design and Gameplay



8.1 Story and objective: The player is trapped inside a mysterious facility and must explore its rooms to collect evidence and a security keycard. The goal is to reach the exit while avoiding the creature. Finding secret evidence changes the available outcome.

8.2 Rooms: The world contains four connected rooms. Side doors allow the player to travel between rooms. Each room can contain objects or hazards that contribute to exploration.

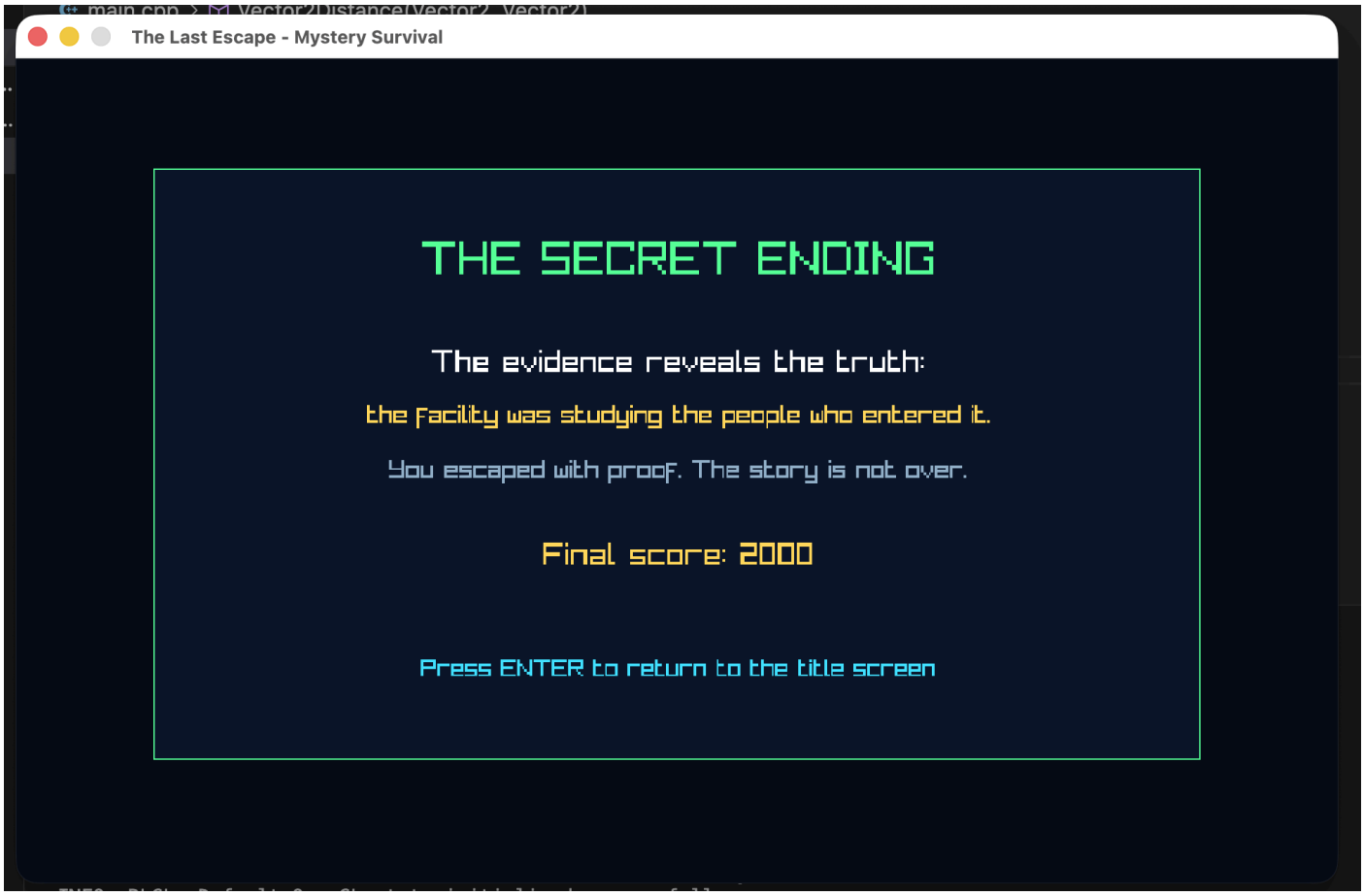
8.3 Collectibles: The player can collect clues, a keycard, a flashlight, and other useful items. An on-screen prompt indicates when an item is close enough to interact with.

8.4 Enemy: A red creature pursues the player in its room. Contact can reduce player health, creating a reason to move carefully and use sprinting when necessary.

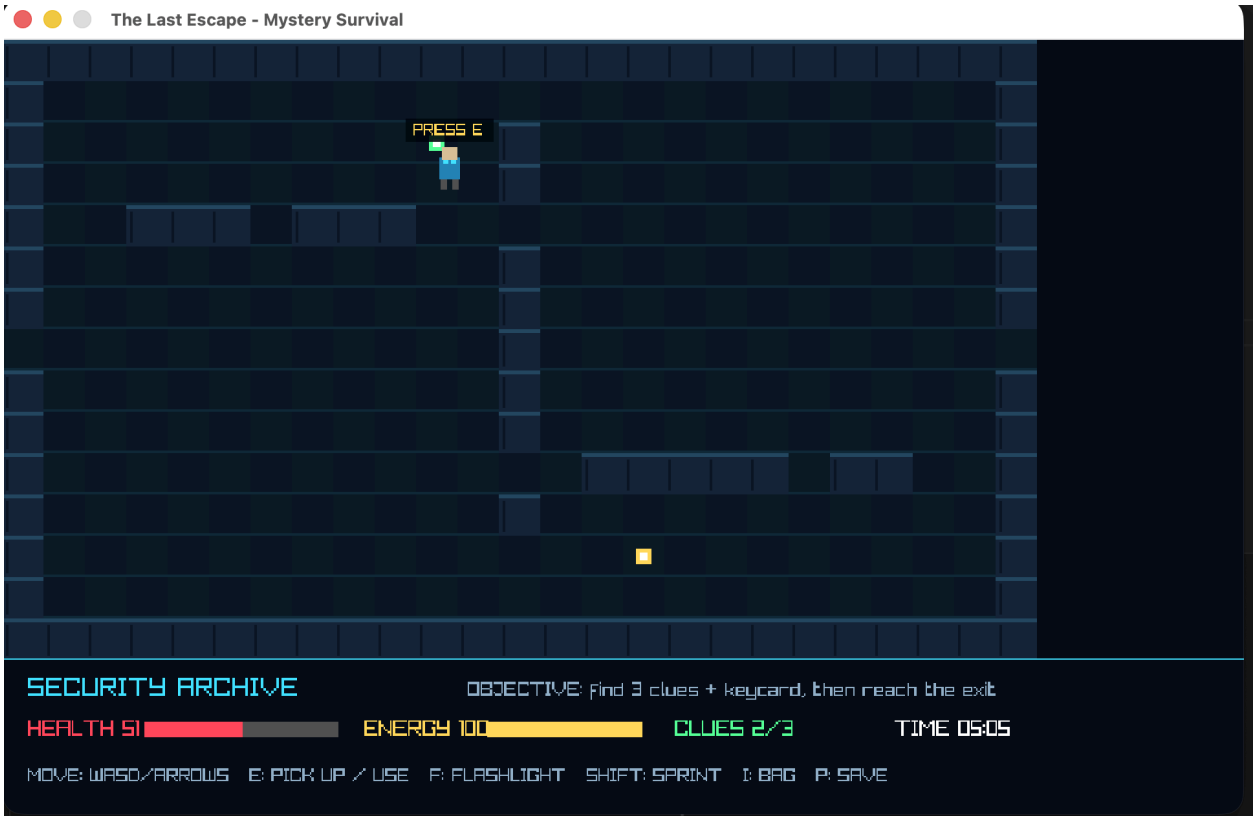
8.5 Player status: Health and energy are tracked. Sprinting consumes energy, while the flashlight can be toggled after it has been collected.

8.6 Outcomes: The game supports an escape ending, a caught ending, and a secret ending. The secret route depends on collecting hidden evidence in addition to the main objectives.




The Last Escape - Mystery Survival
THE SECRET ENDING
The evidence reveals the truth:
the facility was studying the people who entered it.
You escaped with proof. The story is not over.
Final score: 2000
Press ENTER to return to the title screen

9. Controls



Title menu: `1` New Game; `2` Load Game; `3` How to Play; `4` My Records; `5` Quit.

Movement: `W`, `A`, `S`, `D` or the arrow keys.

Interaction: `E` to collect a nearby glowing item or attempt the exit.

Flashlight: `F` to toggle it after collecting it.

Sprint: `Shift` (uses energy).

Inventory: `I`.

Save: `P`.

Menu/navigation: `ESC` to return to the title menu or leave the inventory screen.

10. Object-Oriented Programming Concepts

Data organization: the source code defines separate structures named `Player`, `Enemy`, `Item`, and `Room`. Each groups the data needed to represent a game entity or location.

Encapsulation and modularity: player, enemy, item, and room information is grouped by purpose, while separate functions handle operations such as starting a game, building the world, checking walls, interacting with items, updating game logic, drawing screens, and saving/loading data.

Abstraction: gameplay code calls higher-level functions such as `StartGame()`, `Interact()`, `UpdateGame()`, `DrawRoom()`, `SaveGame()`, and `LoadGame()` rather than repeating all low-level operations at each call site.

State management: an enumeration represents the current screen (title, playing, inventory, help, records, or ending), and another represents the ending state. This keeps screen transitions and outcomes organized.

Note: the current code uses C++ structures and modular functions; it is not a fully class-heavy design. A future version could convert entities into classes with private data and public methods to demonstrate encapsulation and inheritance more explicitly.

11. System Workflow / Algorithm

Initialize the Raylib window and game resources.

Display the title menu and wait for the player's selection.

Start a new game or load saved progress.

Read keyboard input and update player movement.

Check walls and room boundaries to handle collision and room transitions.

Detect nearby items and show an interaction prompt.

When the player presses `E`, collect an item or attempt to use the exit.

Update enemy movement, player health, energy, and gameplay timers.

Draw the current room, player, enemy, items, and HUD.

10. Save progress when requested and change to the appropriate ending when a win or loss condition is met.

12. Implementation Details

The game is implemented in a single `main.cpp` file. Raylib handles the graphical window, drawing primitives, text, keyboard input, frame timing, and platform initialization. The source separates major tasks into functions, including `BuildWorld()`, `IsWall()`, `CollidesAt()`, `StartGame()`, `SaveGame()`, `LoadGame()`, `SaveRecord()`, `Finish()`, `DrawRoom()`, `DrawHUD()`, `Interact()`, `UpdateGame()`, and screen-drawing functions.

The game loop repeatedly checks input, updates the current state, and draws the appropriate screen until the window is closed or the player chooses to quit. Save and record information is stored in local text files in the program's working directory.

13. Build and Execution Procedure

Install dependencies: `brew install raylib pkg-config`.

Open the extracted `the_last_escape_raylib` folder in Visual Studio Code.

Open Terminal → New Terminal and ensure the terminal is in the folder containing `main.cpp`.

Compile with: `g++ main.cpp -std=c++17 -o the_last_escape $(pkg-config --cflags --libs raylib)`.

Run with: `./the_last_escape`.

The initial build was verified on the development Mac after adding a small `Vector2Distance` helper and including `<cmath>`.

14. Testing

The following test cases can be used during demonstration. Mark each result after repeating the test on the final version that will be submitted.

1	Compile the source using g++ and Raylib flags	Program builds without compiler errors	To be verified
2	Launch the executable	Raylib window and title menu appear	Passed: window initialized
3	Start a new game using `1`	Gameplay screen opens	To be verified
4	Move with WASD / arrow keys	Player moves and room boundaries are respected	To be verified
5	Approach a collectible and press `E`	Item is collected and inventory/state updates	To be verified
6	Open inventory using `I`	Inventory screen appears	To be verified
7	Press `P` to save, then load from title menu	Saved progress is restored	To be verified
8	Collect required clues and keycard, reach final exit	Escape outcome is triggered	To be verified
9	Collect hidden evidence and complete objective	Secret ending can be triggered	To be verified
10	Encounter the creature	Player can take damage / caught outcome can occur	To be verified

Testing note: The initial Raylib window initialization and compilation were confirmed from the development terminal output. Update remaining statuses after testing the final build.

15. Limitations

The game currently uses drawn shapes and text rather than detailed external sprite sheets. Its rooms and puzzles are intentionally simple, and the enemy behaviour is basic. Sound effects, background music, animated character sprites, advanced lighting, more varied enemy behaviour, and more elaborate puzzles are not included in the current prototype. Save files are local to the working directory, so moving or deleting those files may affect saved progress.

16. Future Enhancements

Add animated pixel-art sprites for the player and creature.

Add background music, footsteps, door sounds, and jump-scare audio.

Create unique puzzles for each room, including a keypad and collectible code fragments.

Add hiding spots, enemy patrol patterns, and improved enemy AI.

Improve lighting effects and create a flashlight cone or limited visibility.

Add more rooms, dialogue, story notes, and branching decisions.

Strengthen the OOP design by using classes, private members, constructors, and methods for Player, Enemy, Item, and Room.

Add automated tests for inventory, saving/loading, and ending conditions.

17. Conclusion

“The Last Escape” demonstrates how C++ and Raylib can be used to create an interactive graphical game. The project combines movement, collision checks, collectibles, enemy behaviour, player status, screen navigation, persistence, and multiple outcomes. It also provides a practical context for discussing OOP-oriented data organization and modular programming. The current version meets the core goal of moving beyond a terminal-only program and provides a foundation for future visual and gameplay improvements.

18. References

Raylib official website and examples: https://www.raylib.com/

Raylib GitHub repository: https://github.com/raysan5/raylib

C++ reference: https://en.cppreference.com/

Homebrew documentation: https://docs.brew.sh/